MSML 605

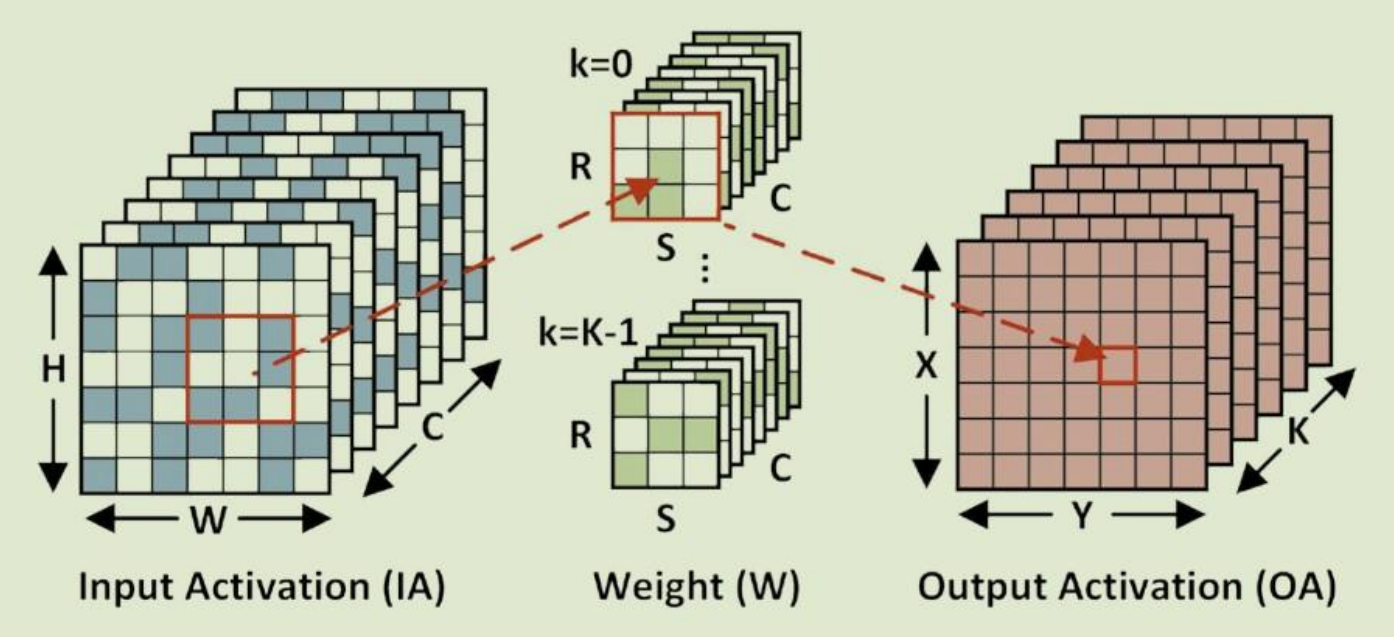
DNN & Hardware - Sparsity

Ref: Machine Learning Hardware Design for Efficiency, Flexibility, and Scalability Jie-Fan Zhang and Zhengya Zhang

Sparse Architecture - Sparsity in NN

- Network sparsity arises from both the model's weights (Ws) and input activations (IAs).
- Techniques like network pruning and other methods of sparsification can be employed to eliminate a significant number of weights from a model, resulting in only a minor decrease in inference accuracy.
- Regarding input activations, certain frequently utilized operators like the rectifier linear unit (ReLU) can constrain all negative activations to zero, leading to sparsity in output activations (OAs), which then serve as input activations (IAs) for the subsequent layer.

CONV computation



Benefits of exploiting sparsity

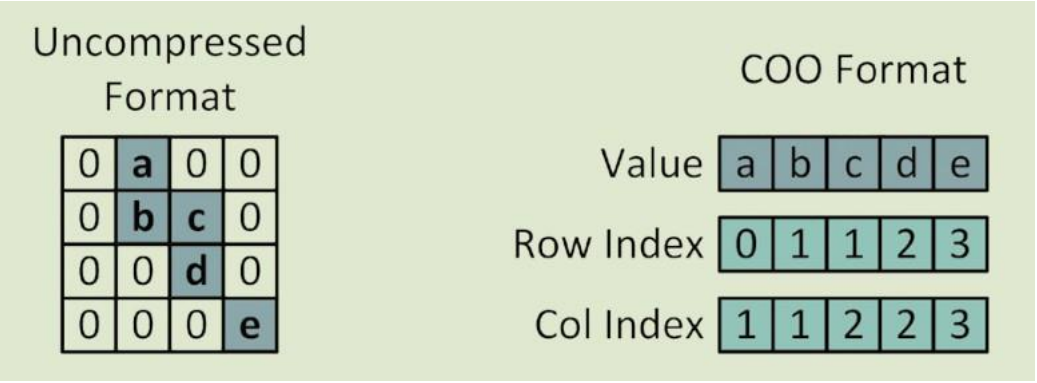
- Data sparsity can be exploited to save power.
- Second, data sparsity can be used to reduce off-chip memory storage and bandwidth usage.
- data sparsity can be used to reduce latency by skipping the ineffectual computation.

Sparse Compression Format

- Sparse compression formats are utilized to compactly store sparse data, aiming to conserve storage space.
- A compressed format contains only nonzero data values and metadata to hold the information for locating the positions of nonzero values in the uncompressed vectors and matrices.

Common Sparse compression formats

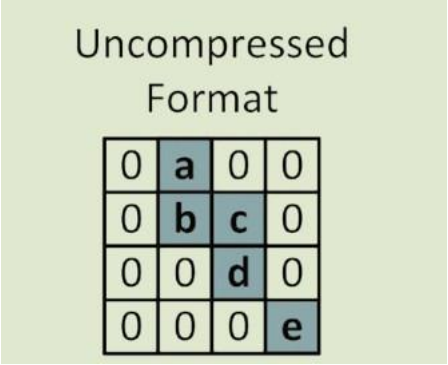
- Coordinate List (COO): In the COO format, nonzero data are stored along with their absolute indices in the original uncompressed vector or matrix

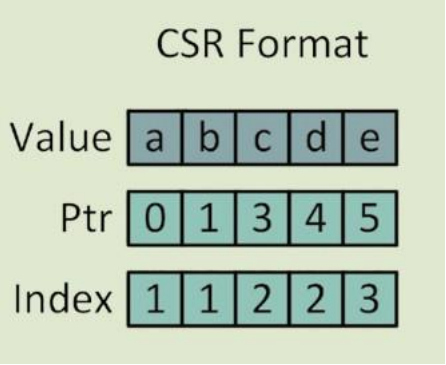


- The advantage of COO is its low decoding complexity, since the row and column indices can be directly used to locate the positions of the nonzero data in the uncompressed vector or matrix.
- However, the row and column indices may require significant amount of storage overhead which makes COO less efficient for data of medium or low sparsity.

Compressed Sparse Row (CSR)

- CSR: Nonzero data are stored first by row, then by column in a 1D value array. Metadata consists of a pointer (Ptr) array and a column index array.

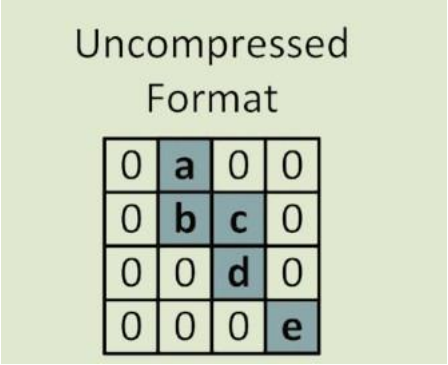


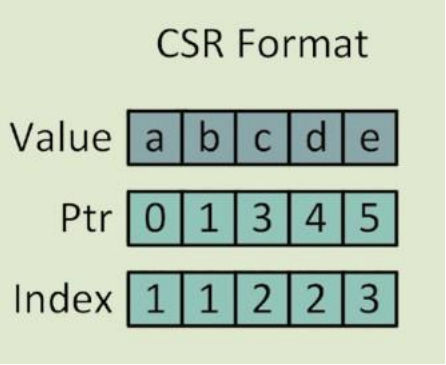


- The Ptr array stores the row-by-row count of the total number of nonzero data. The first entry Ptr[0] is always 0.
- The second entry Ptr[1] stores the count of non-zero data in the first row.

Compressed Sparse Row (CSR)

- CSR: Nonzero data are stored first by row, then by column in a 1D value array. Metadata consists of a pointer (Ptr) array and a column index array.

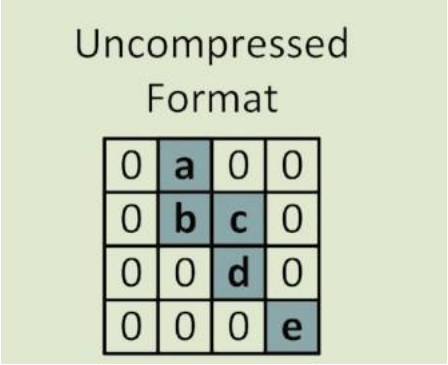


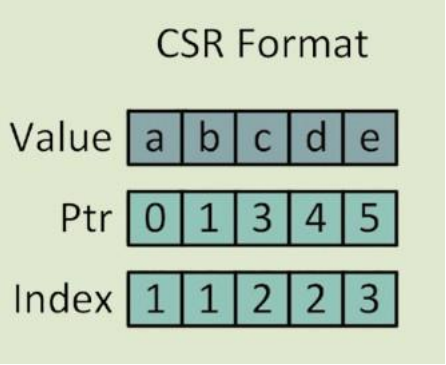


- The second entry Ptr[1] stores the count of nonzero data in the first row.
- Ptr[2] stores the count of nonzero data in the first two rows, etc.
- The column index array stores the column index of each nonzero data.

Compressed Sparse Row (CSR)

- CSR: Nonzero data are stored first by row, then by column in a 1D value array. Metadata consists of a pointer (Ptr) array and a column index array.

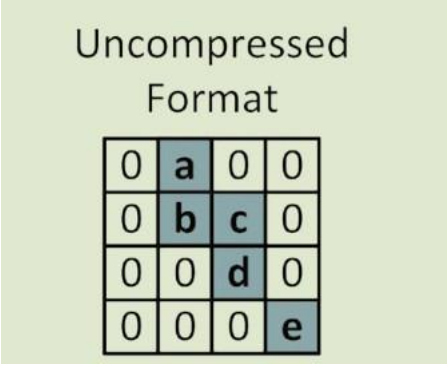


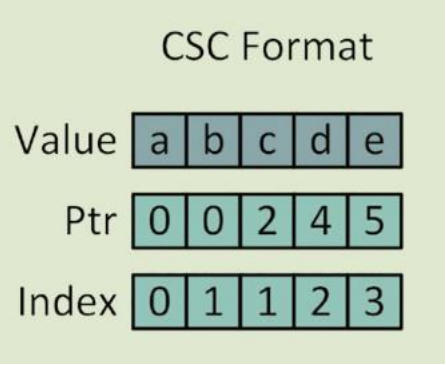


- The decoding of the data is a two-step process.
- To access data in Row 1,
  - Obtain the positions of the nonzero data of Row 1 stored in the value array: Ptr[1], Ptr[1] + 1
  - Obtain the column indices of the nonzero data in Row 1: Index[Ptr[1]], Index[Ptr[1]+1], and so on.

Compressed Sparse Column (CSC)

- CSC: Similar to CSR format, but nonzero data are first stored by column, then by row in a 1D value array.

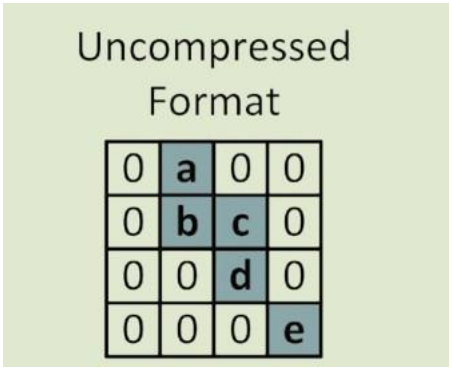


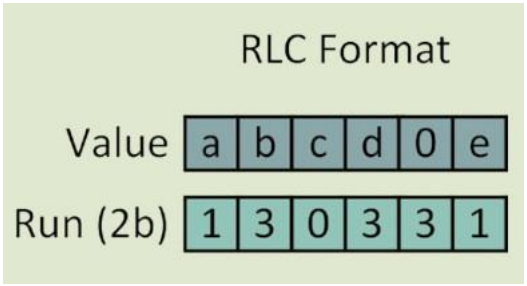


- Ptr stores column-by-column count of the total number of nonzero data.
- Row index array stores the row index of each nonzero data.
- Same advantages and disadvantages as CSR format.

Run_Length Coding (RLC)

- RLC: Nonzero data are stored in a 1D value array in either row or column major.
- Run array keeps track of zeros before each nonzero data, based on run length.

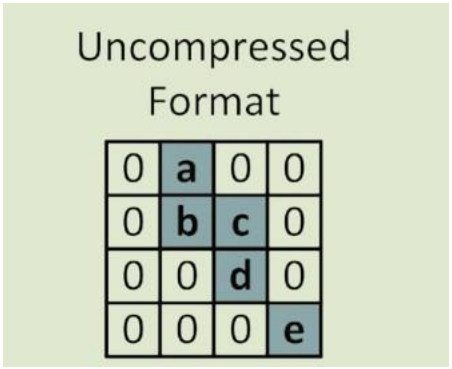


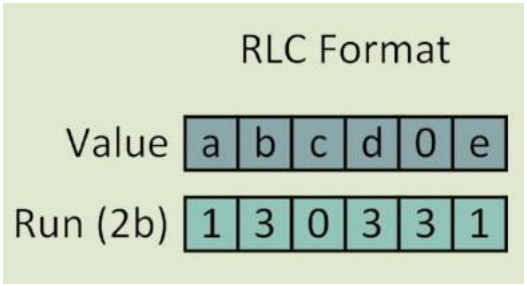


- RLC format shows 2-bit run length.
- Nonzero values, a, b, c, and d are stored in the 1D value array, and they have 1, 3, 0, and 3 preceding zeros or run lengths, respectively.
- Nonzero data, e, has four preceding zeros.

Run_Length Coding (RLC)

- RLC: Nonzero data are stored in a 1D value array in either row or column major.
- Run array keeps track of zeros before each nonzero data, based on run length.

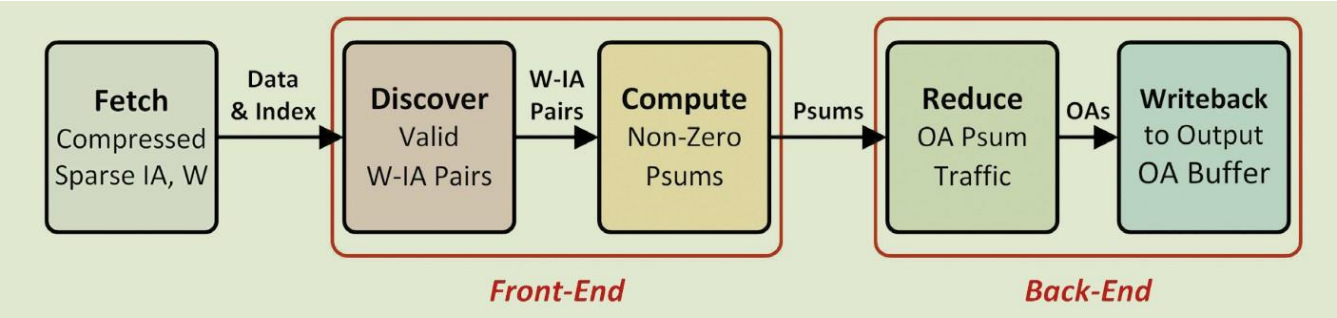




- Nonzero data, , has four preceding zeros, which exceeds the two bits allocated to a run length.
- Additional padding zero is inserted before with a run length of 3.
- Decoding happens in one step.
- Position of the -th nonzero data in the value array can be calculated by accumulating all preceding run lengths in the run array.

Sparse Computation Pipeline

- Nonzero data and metadata arrays of Ws and IAs are first fetched on-chip for processing.
- The compressed sets of W and IA are subsequently examined, matched, and sent to a multiplier array for processing within the initial stage of the pipeline, known as the frontend.



- The calculated partial sums are gathered and stored back into their corresponding output arrays within output buffers in the backend section of the pipeline..
- Challenges:
  - A sufficient number of IA-W pairs must be discovered and sent to the compute stage in order to maintain a high compute utilization. - frontend
  - The irregular sum traffic out of the compute stage must be reduced and written back to the output buffer without costing excessive bandwidth.